S07 · Seguridad distribuida y control de acceso

Jannys Graciela Navarro Acrota

Equipo de referencia: 09 · Casa o Nada. Tarea personal sobre Pagatu.

Aporte documentado: evolución de órdenes, cliente y pago, seguridad, comunicación, verificación y documentación individual con asistencia de herramientas.

Ejecución real: 5 de noviembre de 2026 · America/Lima. Fechas ISO en UTC pueden figurar como 6 de noviembre.

Repositorio GitHub · Stephmarc/Tareas-Distribuidas

Evidencia técnica · cuatro bloques

1. Auth propio construido

Se arrancó el perfil s07 y PostgreSQL auth. El registro real respondió 201, el login con contraseña incorrecta 401 y el correcto 200. La firma RS256 se verificó con el JWKS público. Se consultaron usuarios, roles y usuario_roles y se comprobó el hash BCrypt del usuario de prueba. El JWT íntegro se conserva solo en memoria y no se publica.

2. Gateway Resource Server

La matriz probada con el proveedor final Keycloak devuelve 401 sin token, 401 con firma alterada, 403 al CLIENTE que lista clientes y 200 al ADMIN. Estas verificaciones pertenecen a la evolución final S07b; no se afirma que un token de auth propio y uno de Keycloak compartan emisor. También se repitieron estos casos usando el JWT de auth propio con los Resource Servers reiniciados temporalmente al issuer 8085: 401, 401, 403 y 200, además de crear la orden 52 con clienteId 3 y total 31.00. Al finalizar se restauró Keycloak en todos ellos.

3. Órdenes e identidad firmada

El request no declara clienteId. La respuesta toma clienteId del sub firmado. La llamada directa a órdenes sin JWT devuelve 401 y consultar una orden ajena devuelve 403.

4. Cliente protegido

El servicio cliente valida JWT por su cuenta. La ruta de clientes existe en Gateway. Se probaron ADMIN permitido, CLIENTE denegado y token alterado directamente al puerto 8083. Todas las operaciones CRUD de clientes son administrativas en esta implementación; no existe una ruta de perfil propio.

Matriz: clientes por Gateway o puerto directo: sin JWT/alterado 401, CLIENTE 403, ADMIN permitido. Órdenes: autenticado solo ve las propias; ADMIN puede consultar todas.

Código real del repositorio

AuthController.java

Services/pagatu-auth-ms/src/main/java/pe/edu/upeu/auth/AuthController.java

```
40   @PostMapping("/api/v1/auth/register")
41   @ResponseStatus(HttpStatus.CREATED)
42   @Transactional
43   public Map<String, Object> register(@Valid @RequestBody Credenciales c) {
44     if (
45       usuarios.findByUsername(c.username()).isPresent()
46     ) throw new ResponseStatusException(HttpStatus.CONFLICT);
47     Usuario u = new Usuario();
48     u.username = c.username();
49     u.password = passwords.encode(c.password());
50     u.roles.add("CLIENTE");
51     usuarios.save(u);
52     return Map.of("id", u.id, "username", u.username);
53   }
54
55   @PostMapping("/api/v1/auth/login")
56   public Map<String, Object> login(@Valid @RequestBody Credenciales c) {
57     Usuario u = usuarios
58       .findByUsername(c.username())
59       .filter(v -> passwords.matches(c.password(), v.password))
60       .orElseThrow(() -> new ResponseStatusException(HttpStatus.UNAUTHORIZED));
61     Instant now = Instant.now();
62     var claims = JwtClaimsSet.builder()
63       .issuer("http://localhost:8085")
64       .subject(u.id.toString())
65       .issuedAt(now)
66       .expiresAt(now.plusSeconds(900))
67       .claim("idCliente", u.id.toString())
68       .claim("realm_access", Map.of("roles", u.roles))
69       .build();
70     String token = encoder
71       .encode(JwtEncoderParameters.from(claims))
72       .getTokenValue();
73     return Map.of(
74       "access_token",
75       token,
76       "token_type",
77       "Bearer",
78       "expires_in",
79       900
80     );
81   }
82
83   @GetMapping("/oauth2/jwks")
84   public Map<String, Object> jwks() {
85     return new JWKSet(key.toPublicJWK()).toJSONObject();
86   }
87
88   @GetMapping("/.well-known/openid-configuration")
89   public Map<String, Object> discovery() {
90     return Map.of(
91       "issuer",
92       "http://localhost:8085",
93       "jwks_uri",
94       "http://localhost:8085/oauth2/jwks"
95     );
96   }
97 }
```

SecurityConfiguration.java

Infra/pagatu-gateway/src/main/java/pe/edu/upeu/gateway/SecurityConfiguration.java

```
15     JwtAuthenticationConverter c = new JwtAuthenticationConverter();
16     c.setJwtGrantedAuthoritiesConverter(jwt -> {
17       Map<String, Object> a = jwt.getClaim("realm_access");
18       List<?> roles =
19         a == null ? List.of() : (List<?>) a.getOrDefault("roles", List.of());
20       return roles
21         .stream()
22         .map(
23           r ->
24             (org.springframework.security.core.GrantedAuthority) new SimpleGrantedAuthority(
25               "ROLE_" + r
26             )
27         )
```

```
        .toList();
    });
    return h
      .csrf(s -> s.disable())
      .cors(s -> {})
      .authorizeExchange(a ->
        a
          .pathMatchers(org.springframework.http.HttpMethod.OPTIONS, "/**")
          .permitAll()
          .pathMatchers("/actuator/health/**", "/actuator/prometheus")
          .permitAll()
          .pathMatchers("/api/v1/clientes/**")
          .hasRole("ADMIN")
          .pathMatchers("/api/**")
          .hasAnyRole("ADMIN", "CLIENTE")
          .anyExchange()
          .denyAll()
      )
      .oauth2ResourceServer(o ->
        o.jwt(j ->
          j.jwtAuthenticationConverter(
            new ReactiveJwtAuthenticationConverterAdapter(c)
          )
        )
      )
      .build();
  }
}
```

SecurityConfiguration.java

Services/pagatu-cliente-ms/src/main/java/pe/edu/upeu/cliente/SecurityConfiguration.java

```
package pe.edu.upeu.cliente;

import java.util.*;
import org.springframework.context.annotation.*;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.oauth2.server.resource.authentication.*;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfiguration {

  @Bean
  SecurityFilterChain api(HttpSecurity http) throws Exception {
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(jwt -> {
      Map<String, Object> access = jwt.getClaim("realm_access");
      List<?> roles =
        access == null
          ? List.of()
          : (List<?>) access.getOrDefault("roles", List.of());
      return roles
        .stream()
        .map(r -> new SimpleGrantedAuthority("ROLE_" + r))
        .map(a -> (org.springframework.security.core.GrantedAuthority) a)
        .toList();
    });
    return http
      .csrf(c -> c.disable())
      .authorizeHttpRequests(a ->
        a
          .requestMatchers("/actuator/health/**", "/actuator/prometheus")
          .permitAll()
          .requestMatchers("/api/v1/clientes/**")
          .hasRole("ADMIN")
          .anyRequest()
          .hasRole("ADMIN")
      )
      .oauth2ResourceServer(o ->
        o.jwt(j -> j.jwtAuthenticationConverter(converter))
      )
      .build();
  }
}
```

Ejecución y resultados originales

Comando reproducible: & .\scripts\Run-PersonalEvidence.ps1 -Sesion s07

Los transcripts son salida real de PowerShell y los JSON provienen de las respuestas de la aplicación. Se adjuntan completos en MkDocs. No se sustituyen por un texto de aprobación.

Cuenta tecnica CLIENTE exclusiva, sin rol ADMIN | {"client": "evidencia-personal-cliente"}

Gateway sin token | {"url": "http://localhost:18080/api/v1/ordenes", "status": 401}

Gateway firma alterada | {"url": "http://localhost:18080/api/v1/ordenes", "status": 401}

Gateway CLIENTE a clientes | {"url": "http://localhost:18080/api/v1/clientes", "status": 403}

Gateway ADMIN a clientes | {"url": "http://localhost:18080/api/v1/clientes", "status": 200}

Cliente-ms directo firma alterada | {"url": "http://localhost:8083/api/v1/clientes", "status": 401}

Cliente-ms directo CLIENTE | {"url": "http://localhost:8083/api/v1/clientes", "status": 403}

Orden-ms directo sin token | {"url": "http://localhost:8082/api/v1/ordenes", "status": 401}

POST orden autenticada | {"status": 201, "request": {"metodoPago": "YAPE_PLIN", "detalles": [{"idProducto": 1, "cantidad": 2}]}, "response": {"id": 44, "nombre": "Orden de 9dca6917-56f0-4abc-9f00-3b0495848e74", "descripcion": null, "clienteId": "9dca6917-56f0-4abc-9f00-3b0495848e74", "estado": "PENDIENTE_PAGO", "total": 31, "detalles": [{"id": 44, "nombreProducto": "Producto de prueba", "productoId": 1, "cantidad": 2, "precioUnitario": 15.5, "ordenId": 44}]}, "trace": "personal-s07-d76bb045-774b-4797-835d-1cd54f5ba794"}

Identidad procede de sub firmado, sin clienteId en request | {"subject": "9dca6917-56f0-4abc-9f00-3b0495848e74", "clienteIdRespuesta": "9dca6917-56f0-4abc-9f00-3b0495848e74"}

POST orden autenticada | {"status": 201, "request": {"metodoPago": "YAPE_PLIN", "detalles": [{"idProducto": 1, "cantidad": 2}]}, "response": {"id": 45, "nombre": "Orden de 891b60fa-0606-48a7-bd4c-7ef9ed5eabdf", "descripcion": null, "clienteId": "891b60fa-0606-48a7-bd4c-7ef9ed5eabdf", "estado": "PENDIENTE_PAGO", "total": 31, "detalles": [{"id": 45, "nombreProducto": "Producto de prueba", "productoId": 1, "cantidad": 2, "precioUnitario": 15.5, "ordenId": 45}]}, "trace": "personal-s07-df67c656-f897-4b7f-b7cd-da54164bf05f"}

CLIENTE no consulta orden de otro usuario | {"status": 403, "ordenAjena": 45}

Registro HTTP 201 | {"status": 201, "data": {"username": "personal-1793935617701", "id": 2}}

Login incorrecto HTTP 401 | {"status": 401, "data": {"timestamp": "2026-11-06T03:26:58.218Z", "status": 401, "error": "Unauthorized", "path": "/api/v1/auth/login"}}

Login correcto HTTP 200 | {"status": 200, "token_type": "Bearer", "expires_in": 900, "nota": "JWT completo omitido para no publicar credenciales"}

Firma RS256 verificada con JWKS publico | {"header": {"kid": "27beea61-afa5-4ed4-97c0-ed3f67662a79", "alg": "RS256"}, "claims": {"iss": "http://localhost:8085", "sub": "2", "idCliente": "2", "realm_access": {"roles": ["CLIENTE"]}, "exp": 1793936518, "iat": 1793935618}, "jwksPublico": "Clave pública completa y resultado de verificación en s07-auth.json"}

Tablas y hash BCrypt real de usuario exclusivo | {"usuarios": "2|personal-1793935617701|$2a$10$.uLJUGyNQ5wiIlB1VC4S..9xLLzqQz9zUaHFH3aLPWaCd0Icbfipe", "roles": "ADMIN\nCLIENTE", "usuario_roles": "2|CLIENTE"}

JWT propio: Registro propio | {"status": 201, "usuario": {"username": "legacy-evidencia-1793936547203", "id": 3}}

JWT propio: Sin JWT | {"status": 401, "url": "http://localhost:18080/api/v1/ordenes", "issuer": "http://localhost:8085"}

JWT propio: JWT alterado | {"status": 401, "url": "http://localhost:18080/api/v1/ordenes", "issuer": "http://localhost:8085"}

JWT propio: CLIENTE no es ADMIN | {"status": 403, "url": "http://localhost:18080/api/v1/clientes", "issuer": "http://localhost:8085"}

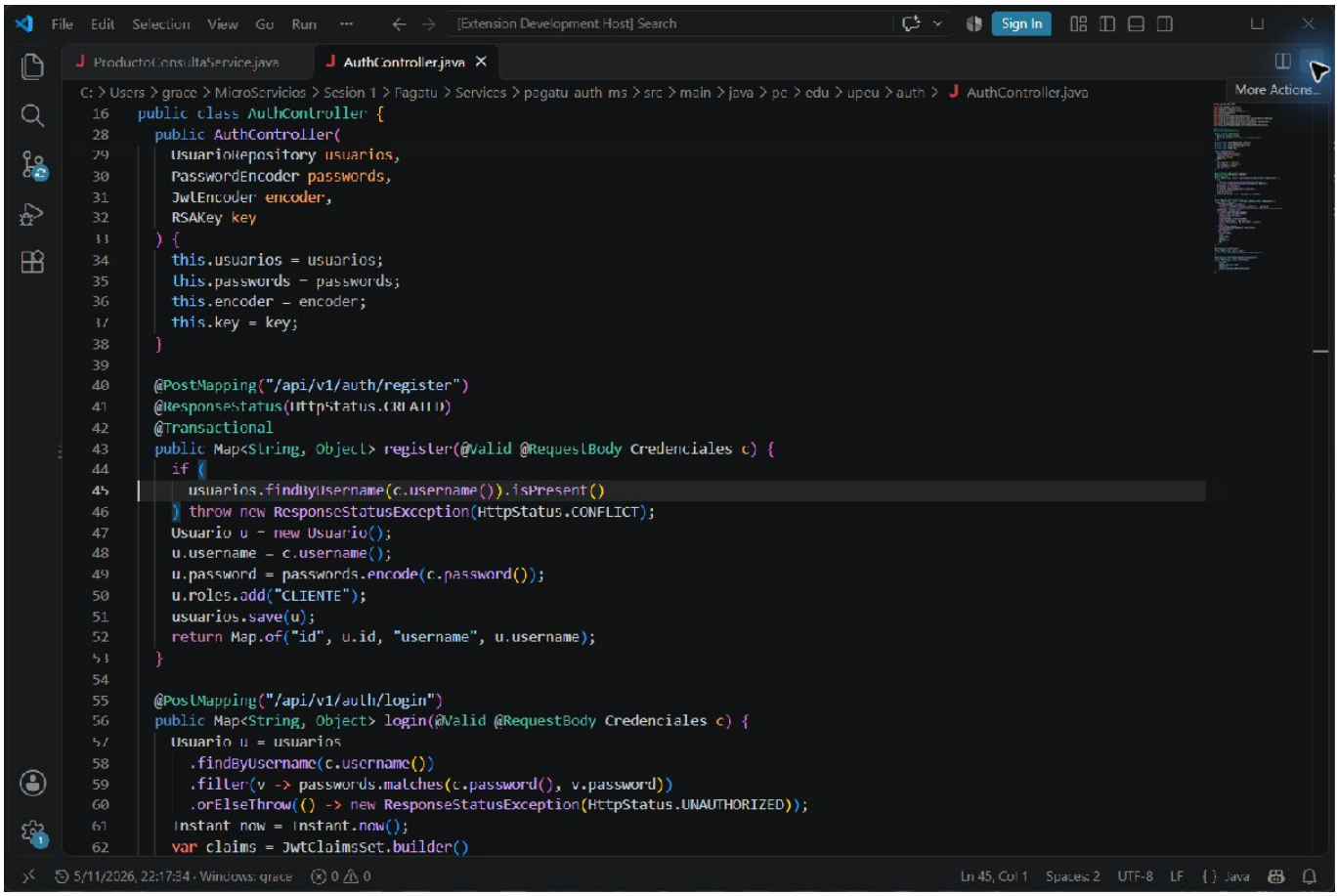
JWT propio: Cliente directo JWT alterado | {"status": 401, "url": "http://localhost:8083/api/v1/clientes", "issuer": "http://localhost:8085"}

JWT propio: Orden directa sin token | {"status": 401, "url": "http://localhost:8082/api/v1/ordenes", "issuer": "http://localhost:8085"}

JWT propio: Orden con identidad del JWT propio y precio Feign | {"request": {"metodoPago": "YAPE_PLIN", "detalles": [{"idProducto": 1, "cantidad": 2}]}, "response": {"id": 52, "nombre": "Orden de 3", "descripcion": null, "clienteId": "3", "estado": "PENDIENTE_PAGO", "total": 31, "detalles": [{"id": 52, "nombreProducto": "Producto de prueba", "productoId": 1, "cantidad": 2, "precioUnitario": 15.5, "ordenId": 52}]}, "status": 201}

JWT propio: ADMIN propio permitido | {"status": 200, "issuer": "http://localhost:8085", "nota": "Rol agregado solo al usuario nuevo exclusivo de esta prueba"}

Captura real



Captura íntegra, sin edición de resultados ni fechas. S07 auth codigo. La salida original completa está vinculada en MkDocs.

Hallazgo y reflexión técnica

Error o hallazgo real

Auth y Keycloak tienen distintos emisores y claves. Las pruebas separan la emisión RS256 de S07 del Resource Server final S07b; mezclar sus tokens causaría 401. También se corrigió el archivo de log de auth, que apuntaba erróneamente a cliente.log.

Reflexión técnica

Los permisos del Gateway y del servicio siguen expresándose mediante roles.

La entidad de órdenes sigue tomando su identidad de un claim firmado.

Al migrar cambia el issuer y la ubicación del JWKS, no la regla de negocio.

La clave privada permanece en el servidor de autorización.

Los Resource Servers verifican con claves públicas sin poder emitir tokens.

Keycloak administra usuarios y sesiones que antes resolvía auth propio.

En esta tarea clienteId representa quién compra y coincide con sub; no requiere mapper adicional.

Límite de alcance: esta entrega personal usa Pagatu; la adaptación al Proyecto Sello grupal CASAS se evalúa por separado. No se atribuye una defensa oral realizada ni una nota de evaluación.

Anexo · Feedback de la sesión

Campos personales pendientes de completar por Jannys. No se inventan respuestas sobre la experiencia, comprensión, esfuerzo o satisfacción en clase.

1. ¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?

2. ¿Qué punto de la clase te resultó más confuso o te dejó con dudas?

3. ¿Tienes alguna pregunta para la siguiente clase?

4. Nivel de comprensión: Entendido / Más o menos / Necesito ayuda.

5. ¿Cómo puede ayudarte el docente a comprender mejor el tema?

6. Participación: Muy comprometido/a / Comprometido/a / Poco comprometido/a.

7. Satisfacción con la clase: 1 a 10.